COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

07

PACKAGE 8 · HANDBOOK

Computer Engineering and Digital Design

Expanded teaching handbook · Lessons 31.1-35.4

Undergraduate computer-engineering core

Modules 31-35 · Gate G

Edition: September 28, 2026

A guided self-study curriculum. Supplied examples are learning references, not completed learner projects. Simulation is the default; physical hardware is optional.

Contents

Page references and PDF bookmarks are linked for navigation.

How to read a lesson 3

Module 31 - Circuits, measurement and electronic foundations 4

Lesson 31.1 - Voltage, current, resistance and safe measurement 4

Lesson 31.2 - Capacitors, inductors, transients and frequency response 6

Lesson 31.3 - Diodes, transistors, CMOS and analog interfaces 8

Lesson 31.4 - Sampling, ADCs, DACs and noisy controller inputs 10

Module 32 - Digital logic, state and timing 13

Lesson 32.1 - Boolean logic and combinational building blocks 13

Lesson 32.2 - Registers, finite-state machines and synchronous timing 15

Lesson 32.3 - Binary arithmetic, fixed point and overflow policies 17

Lesson 32.4 - Clock domains, reset and metastability 19

Module 33 - SystemVerilog and verification-driven RTL design 22

Lesson 33.1 - RTL semantics and synthesizable SystemVerilog 22

Lesson 33.2 - Testbenches, scoreboards and constrained stimulus 24

Lesson 33.3 - Assertions, coverage and formal reasoning 26

Lesson 33.4 - Synthesis, constraints and FPGA resource mapping 29

Module 34 - Designing a small processor 32

Lesson 34.1 - Instruction sets, datapaths and single-cycle execution 32

Lesson 34.2 - Pipelining, forwarding, stalls and branch recovery 34

Lesson 34.3 - Memory interfaces, caches and protection boundaries 36

Lesson 34.4 - Processor verification, toolchains and architectural evidence 39

Module 35 - FPGA-based game subsystems and integration 42

Lesson 35.1 - Memory-mapped I/O, timers, interrupts and DMA 42

Lesson 35.2 - Framebuffers, scanout and pixel pipelines 44

Lesson 35.3 - A small accelerator and its software contract 47

Lesson 35.4 - Integrated digital-design review and optional board bring-up 49

Reference directory 51

Package 8 | Undergraduate CE core | Modules 31-35 | Gate G

Move from ideal circuits and logic to verified RTL, a processor and an integrated game-oriented subsystem. This expanded volume preserves every original lesson identifier and requirement. It contains 20 original expanded teaching chapters, the 60 original core quiz questions and 40 new transfer questions.

Use the separate workbook for full lab procedures and module-project milestones. Answers are in a separate key. Read START_HERE for the tool ladder and completion policy. The authoring tests establish Python reference behavior only; included hardware-tool examples were not executed in the authoring environment.

How to read a lesson

Begin with the game connection and outcome, derive the worked example before running code, then complete the lab and assigned reading. Answer the quiz without the key. Each lesson needs study, practical evidence and at least 5/6 on the core quiz; transfer questions are an additional diagnostic.

Do not confuse an abstract model with physical hardware. Repeated distinctions between acceptance, scheduling, timing and proof are part of the engineering content, not optional caveats. Most lab failures in this volume are failures to state or preserve a contract.

Module 31 - Circuits, measurement and electronic foundations

Prerequisites: Modules 06-10; basic algebra and calculus competence required. Simulation is the default laboratory.

Module project: Create a circuit laboratory notebook using simulation first: voltage/current prediction, an RC transient, a transistor switch and a sampled input. Physical extensions require a qualified mentor and a documented low-voltage safety review.

31.1 - Voltage, current, resistance and safe measurement. Analyze a simple circuit using units and conservation laws.

31.2 - Capacitors, inductors, transients and frequency response. Relate stored energy to timing and filtering behavior.

31.3 - Diodes, transistors, CMOS and analog interfaces. Understand the physical devices beneath digital abstractions.

31.4 - Sampling, ADCs, DACs and noisy controller inputs. Convert physical signals into stable digital measurements.

Lesson 31.1 - Voltage, current, resistance and safe measurement

Learning outcome: Analyze a simple circuit using units and conservation laws.

Game-engineering connection: A controller, graphics card and game console all turn physical voltages into logical decisions. Begin by predicting a circuit before trusting either a simulator or a measurement.

A circuit is a model with a boundary

Voltage is a difference in electric potential, not something that belongs to one isolated wire. Choose a reference node and label every other node relative to it. Current is signed charge flow through a branch. When a calculation returns a negative current, it usually means the actual direction is opposite the arrow you chose; it does not mean that conservation failed.

The lumped-circuit model treats components as localized elements connected by ideal wires. It is useful when propagation effects and distributed fields are small enough for the question being asked. A resistor-divider calculation is not a model of a high-speed display cable. State the model boundary: ideal DC source, ideal resistors, no temperature change, and either no load or a specified load. [R30]

For an ideal resistor, V = I R. Kirchhoff's current law says the signed sum of currents at a node is zero. Kirchhoff's voltage law, under the lumped model, makes the signed voltage changes around a loop sum to zero. These are equations to solve, not rules for guessing whether current “prefers” one branch. Current divides between all conducting parallel paths according to their impedances.

Derive the divider rather than memorize it

Put R1 between a source Vs and an output node, and R2 between that node and ground. With no load, the same current passes through both resistors: I = Vs/(R1 + R2). The output is the voltage across R2, so Vout = Vs R2/(R1 + R2). Check limiting cases: if R2 approaches zero, Vout approaches zero; if R1 approaches zero, Vout approaches Vs.

Now attach a load RL from the output to ground. It is in parallel with R2, giving Rb = R2 RL/(R2 + RL). Replace R2 by Rb in the divider formula. This matters for analog controller inputs: the measurement circuit can change the signal being measured. A divider is not an ideal voltage regulator. The Thevenin equivalent seen at its output has Vth equal to the unloaded voltage and Rth = R1 parallel R2, with an ideal independent voltage source replaced by a short when finding Rth.

Power, tolerance and measurement loading

Power absorbed by a resistor is P = VI = I squared R = V squared/R, using passive sign conventions. Units expose errors: volts divided by ohms gives amperes, not milliamperes. A result of 0.005 A is 5 mA. Component power ratings and recommended operating conditions must be checked separately from the ideal calculation.

Tolerance is an interval, not an automatic probability distribution. For a divider with independent stated bounds, the largest ratio occurs with the bottom resistor high and the top resistor low. A random sample inside those bounds is not a proof that the extreme combinations are safe. Distinguish a worst-case bound from a Monte Carlo estimate; use each for the question it can answer.

An ideal voltmeter has infinite input resistance. A real meter or oscilloscope loads the circuit and has range, reference and grounding constraints. An ammeter belongs in the branch whose current is measured; connecting a low-resistance current input across a source can create a short. In this course, perform the required work in simulation. Any physical extension needs a qualified mentor, reviewed low-voltage supply and wiring plan. Do not open mains-powered equipment, displays or battery packs.

Worked derivation: a loaded controller reference

Let Vs = 5 V, R1 = 2,000 ohms and R2 = 3,000 ohms. Unloaded current is 1 mA and Vout is 3 V. R1 dissipates 2 mW and R2 dissipates 3 mW. The source supplies 5 mW, matching the sum of resistor powers.

With RL = 6,000 ohms, the bottom equivalent becomes 2,000 ohms. The output falls to 2.5 V. Top current is 1.25 mA; R2 current is 0.833333 mA; load current is 0.416667 mA. The node residual is 1.25 - 0.833333 - 0.416667 = 0 mA to rounding. The load-induced drop is 16.6667% relative to the unloaded 3 V, not 10% relative to the source.

For the unloaded divider with 1% resistor bounds, the maximum output is 5(3030)/(1980 + 3030) = 3.023952 V. The minimum is 5(2970)/(2020 + 2970) = 2.975952 V. These are ideal-model extremes with an exact 5 V source. Source tolerance would widen the bound.

Engineer's check

Before simulation, predict the direction of the loading change. After simulation, calculate the KCL residual and source-versus-load power balance. A pretty voltage plot without units, source assumptions or a load description is not engineering evidence. Return to the node equation when a result disagrees, and record the disagreement instead of adjusting the prediction afterward.

Practical checkpoint

Lab 31.1: Loaded-divider notebook. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R30: lumped-circuit abstraction, resistive networks and basic circuit analysis.

Core quiz - original program questions / 6 points

31.1-C1. What current flows through 1,000 ohms at 5 V?

31.1-C2. What power is dissipated in that ideal resistor?

31.1-C3. Why can putting a meter in the wrong current-measurement configuration be dangerous?

Transfer check - new application questions / 4 points

31.1-T1. A 5 V source drives R1=2 kohm above R2=3 kohm. A 6 kohm load is placed across R2. Find Vout and all three branch currents.

31.1-T2. Why does testing 1,000 random resistor values inside a tolerance band not prove the worst-case divider voltage?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 31.2 - Capacitors, inductors, transients and frequency response

Learning outcome: Relate stored energy to timing and filtering behavior.

Game-engineering connection: Filtering a joystick and maintaining a stable power rail both depend on stored energy and time. The same capacitor that smooths noise also delays a useful change.

State comes from stored energy

An ideal capacitor obeys i = C dv/dt and stores energy E = C V squared/2. Its voltage is continuous unless an ideal impulse of current supplies the required charge instantly. An ideal inductor obeys v = L di/dt and stores E = L I squared/2; its current has the analogous continuity constraint. These are state variables: knowing only present source voltage is not enough to predict the next instant without initial capacitor voltage or inductor current. [R30]

For a resistor feeding a capacitor to ground, Kirchhoff's current law gives (Vs - Vc)/R = C dVc/dt. With a constant source after time zero, define tau = RC. Solving the first-order differential equation yields Vc(t) = Vs + (V0 - Vs) exp(-t/tau). This formula handles charging and discharging; setting V0=0 is a special case, not a universal initial condition.

At one tau, a zero-initial-voltage capacitor has traversed 1-exp(-1), about 63.2%, of the change. At three tau, about 95.0%; at five tau, about 99.3%. “Settled” therefore requires a stated tolerance. Reaching exactly the final voltage takes infinite time in this ideal model, so a test that demands exact equality is badly specified.

Time domain and frequency domain answer different questions

A low-pass RC circuit driven by a sinusoid has H(jw) = 1/(1 + jwRC). The magnitude is 1/sqrt(1 + (wRC) squared), and the phase is -atan(wRC). The cutoff frequency is fc = 1/(2 pi RC), where magnitude is 1/sqrt(2), approximately -3.01 dB, and phase is -45 degrees. This is a steady-state frequency response, not a replacement for startup analysis. [R10]

A filter's transient lag is often a user-experience issue. An aggressive low-pass filter can remove visible joystick jitter while making aiming feel late. A power-decoupling capacitor must deliver transient current through real impedance. Equivalent series resistance and inductance, source impedance and load step shape can matter. The ideal RC exercise does not select a production decoupling network or prove stability of a regulator.

Numerical simulation is also a model

Forward Euler updates Vnext = V + dt(Vs - V)/tau. Its step size changes accuracy and can even change stability. For this first-order decay, the homogeneous error multiplier is 1 - dt/tau; absolute stability requires 0 < dt/tau < 2, but a stable result can still be far too inaccurate. With dt greater than tau the numerical response alternates about the final value even though the true RC step is monotonic.

For a piecewise-constant input, use the exact interval update Vnext = Vs + (V - Vs)exp(-dt/tau). That avoids the Euler integration error for this model. It still does not include omitted parasitics or a continuously changing input within the interval. Convergence testing means reducing dt and checking that a quantity of interest approaches a stable value, not simply plotting more points.

Worked derivation: delay and attenuation

With R=10 kohm and C=1 microfarad, tau=0.010 s and fc=15.9155 Hz. A 0-to-3.3 V step reaches 2.0860 V after 10 ms. The time to 90% is -tau ln(0.1)=23.0259 ms; the time to 99% is 46.0517 ms. The capacitor's final stored energy is 0.5(1e-6)(3.3 squared)=5.445 microjoules.

At 100 Hz, wRC=2 pi, giving magnitude about 0.15718 and phase about -80.96 degrees. A 1 V sinusoidal amplitude at the input becomes approximately 0.157 V in steady state under the model. Doubling C doubles tau, halves fc, and increases step-response delay. That tradeoff is not free noise removal.

If the initial voltage is 1 V instead of zero, the value after one tau is 3.3 +(1 - 3.3)/e = 2.4539 V. The same time constant describes a different trajectory because stored energy differs. For an inductor example, 10 mH carrying 100 mA stores 50 microjoules. Interrupting current through a real inductive load can produce a substantial voltage; physical switching experiments need an independently reviewed protection design.

What to carry into digital design

Clock edges are electrical transients, not abstract instants. Later timing models summarize some of their consequences as propagation, setup and hold constraints. Here, practice the discipline of naming the approximation. “The RC model agrees with the equation” is a valid narrow result. “This board will be reliable” is not a conclusion supplied by that result.

Practical checkpoint

Lab 31.2: Transient and frequency-response investigation. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R30: capacitors, inductors, first- and second-order systems; R10: frequency response.

Core quiz - original program questions / 6 points

31.2-C1. What is tau for 10 kohm and 1 microfarad?

31.2-C2. What fraction of a zero-to-final RC charging step is reached at one tau?

31.2-C3. Why do initial conditions matter?

Transfer check - new application questions / 4 points

31.2-T1. For the 10 kohm, 1 microfarad low-pass, find the 90% rise time and cutoff frequency.

31.2-T2. Why is a forward-Euler RC response that overshoots at dt=1.5 tau not evidence that the ideal physical RC circuit overshoots?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 31.3 - Diodes, transistors, CMOS and analog interfaces

Learning outcome: Understand the physical devices beneath digital abstractions.

Game-engineering connection: A logical one is an allowed voltage range, not a universal electrical language. Understanding that distinction prevents a correct-looking interface from damaging or misreading hardware.

The digital abstraction rests on analog behavior

A diode's current-voltage relation is nonlinear. A silicon-diode “0.7 V drop” is an approximate operating-point mnemonic, not a constant valid at all currents and temperatures. A MOS transistor's channel behavior depends on terminal voltages, geometry, process and operating region. Its gate is not a software Boolean variable, and the threshold voltage is not a guarantee of a low-resistance on-state. [R30]

For a switching application, select operating conditions using the device's documented on-resistance or output behavior at the actual gate drive, not the threshold headline. Also consider absolute limits, current, temperature and transient behavior. This lesson uses educational models rather than selecting parts for construction. A model parameter is an assumption unless it comes from an identified, applicable device source.

Build the CMOS inverter mentally

A basic CMOS inverter places a p-channel device between VDD and the output and an n-channel device between the output and ground. Their gates share the input. A low input tends to turn on the pull-up and turn off the pull-down; a high input does the opposite. Around the switching region, both devices may conduct and the output changes continuously. Leakage and charging current mean physical logic is not free of power consumption.

The static transfer curve plots output voltage against input voltage. Digital interface specifications commonly distinguish the maximum accepted low input, VIL, and minimum accepted high input, VIH, from output guarantees VOL and VOH under stated loading. A low noise margin can be written VIL(max)-VOL(max); a high margin is VOH(min)-VIH(min). A positive margin is necessary for the stated static conditions, but it does not establish every transient, loading or voltage-tolerance requirement.

For example, a source guaranteed to output at least 2.4 V does not guarantee a valid high to a receiver requiring at least 2.6 V. Even a source that reaches 5 V may violate a 3.3 V receiver's input limits. Logical compatibility and damage avoidance are different checks. Never assume every input has a useful clamp diode or that an absolute-maximum specification permits normal operation.

Comparators, hysteresis and amplifiers

A comparator decides which of two input voltages is greater, subject to its actual input range, delay and output behavior. Noise near one threshold can make the output chatter. Hysteresis deliberately provides different rising and falling thresholds. The output retains its previous state while the input remains between those thresholds. This is useful for a noisy trigger or button-like analog signal, but it changes when a transition is recognized.

An operational amplifier with negative feedback can provide a predictable closed-loop gain only while the assumptions supporting the feedback solution remain valid. The ideal non-inverting formula is gain = 1 + Rf/Rg. It does not override power-supply rails, input common-mode limits, gain-bandwidth, slew rate, output current or stability constraints. A saturated amplifier is not well modeled by pretending its virtual-short approximation still holds exactly.

Worked example: limits before code

Take a fictional logic interface with VOH(min)=2.9 V, VIH(min)=2.0 V, VOL(max)=0.2 V and VIL(max)=0.8 V under specified conditions. The static high margin is 0.9 V and low margin 0.6 V. Those numbers say nothing about whether a 5 V source exceeds the receiver's absolute input limit; that is another field in the interface review.

For an ideal non-inverting amplifier with Rf=9 kohm and Rg=1 kohm, gain is 10. A 0.2 V input requests 2.0 V output. A 0.5 V input requests 5.0 V; a model powered from 0 to 3.3 V cannot provide that full output, and a real part may have an even narrower swing. The linear gain calculation identifies an invalid operating point, not an achievable output.

Now define a hysteretic comparator: switch high at input >=1.8 V, switch low at input <=1.5 V, otherwise keep the prior state. Starting low, the sequence 1.4, 1.6, 1.7, 1.81, 1.7, 1.6, 1.49 V gives outputs 0,0,0,1,1,1,0. Equality conventions are part of this educational model. A physical component has tolerances and propagation delay instead of exact mathematical thresholds.

Modeling discipline

The supplied CMOS SPICE deck uses simple level-1 device models to make the transfer mechanism visible. It is not a characterized production process and cannot certify a board interface. Compare model curves, operating regions and sensitivity. For a physical extension, replace invented limits with the exact selected component documents and have the wiring and voltage-domain plan reviewed before connection.

Practical checkpoint

Lab 31.3: CMOS transfer and noisy-threshold study. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R30: MOS transistors, amplifiers, digital circuits and operational amplifiers.

Core quiz - original program questions / 6 points

31.3-C1. Why is a transistor not only a binary software switch?

31.3-C2. Does a logical high guarantee compatibility across voltage domains?

31.3-C3. Why distinguish absolute maximum ratings from recommended operating conditions?

Transfer check - new application questions / 4 points

31.3-T1. A non-inverting ideal gain is 10 on a nominal 0-to-3.3 V supply. Does a 0.5 V input establish a valid 5 V output?

31.3-T2. Starting low, a comparator sets at >=1.8 V and clears at <=1.5 V. What outputs follow 1.4, 1.6, 1.81, 1.7, 1.49 V?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 31.4 - Sampling, ADCs, DACs and noisy controller inputs

Learning outcome: Convert physical signals into stable digital measurements.

Game-engineering connection: A high-resolution joystick can still feel inaccurate or slow. Accuracy, quantization, filtering, calibration and latency are distinct parts of the input system.

Sampling and quantization are different operations

Sampling selects values at particular times. Quantization assigns values to a finite set of codes. An N-bit converter supplies 2 to the N distinct codes, but that count is not a guarantee of accuracy. Reference error, offset, noise, nonlinearity, input settling and clock jitter can matter. Resolution describes the coding grid; accuracy describes agreement with the intended quantity. [R10, R30]

Choose a conversion convention explicitly. This volume's simple unipolar model uses codes k=floor(V/Vref times 2^N), clipped to 0 through 2^N-1. Its nominal bin width is Vref/2^N. For reconstructions at bin centers, Vhat=(k+0.5)Vref/2^N gives error bounded by half a step away from clipping under the ideal bin model. A different mapping that puts endpoint codes at exactly 0 and Vref uses 2^N-1 in a scale factor; that is not the same convention. Do not quietly mix them.

Why analog filtering comes first

Uniform sampling cannot distinguish every continuous signal. Frequencies separated by integer multiples of the sample rate can produce the same samples. A component above the intended sampled band may appear inside it as an alias. An analog bandwidth-limiting stage must attenuate unwanted input content before sampling when aliasing is a concern. Digital filtering can remove some sampled frequencies, but cannot generally tell aliased content from genuine content at the same sampled frequency. [R10]

For a band-limited baseband signal, sampling above twice the highest frequency is a starting theorem under ideal conditions, not a complete practical filter design. Real transition bands, noise and sampling uncertainty require margin. A first-order RC filter may provide insufficient attenuation for a given sensor and sample rate; calculate attenuation rather than claiming that the presence of a capacitor solves aliasing.

Calibration, dead zones and filtering

Map a measured center and endpoint range into a normalized controller value. Center offset correction removes systematic resting bias but can shift the available range asymmetrically. A dead zone suppresses small commands near rest. A hard dead zone sets small magnitudes to zero; a rescaled dead zone also remaps the remaining range to avoid a discontinuity in output scale. Document what happens exactly at the boundary.

A simple exponential filter uses y[n]=y[n-1]+alpha(x[n]-y[n-1]), with 0<alpha<=1. For a unit step from zero, y[n]=1-(1-alpha)^n when n counts updates. The filter reduces rapid variation but adds response delay. Debouncing a switch instead classifies unstable transitions over time; it is not interchangeable with filtering a continuously variable joystick.

Latency has a chain: time until the next sample, conversion delay, filter response, software scheduling and the game/render pipeline. For asynchronous arrivals uniformly distributed relative to a sample clock, the mean wait to the next sample is half a sample period; that statement depends on the uniform-arrival assumption. Worst-case waits, filter settling and end-to-end latency are different quantities.

Worked example: quantify the input path

For an ideal 12-bit, 3.3 V converter, the bin width is 3.3/4096 = 0.000805664 V, or 0.805664 mV. There are 4096 codes numbered 0 through 4095. A 2 mV offset is about 2.482 codes: nominal bit count did not make the offset disappear.

At a 1,000 samples/s rate, a 700 Hz cosine produces the same sampled cosine values as a 300 Hz cosine under this phase convention, because 700=1000-300. A sine example may have a sign reversal; specify the waveform and phase rather than comparing frequencies alone. Once the 700 Hz component has become 300 Hz in the samples, an ordinary digital low-pass preserving 300 Hz cannot distinguish it from a genuine 300 Hz signal.

With alpha=0.25 and a 1 ms sample period, the filter reaches at least 90% after n=ceil(ln(0.1)/ln(0.75))=9 updates. After 8 updates the output is about 0.899887; after 9 it is 0.924915. That is 9 ms measured from immediately before the first update under our convention; arbitrary event arrival adds a sample-phase wait. Explain the timing reference whenever presenting a latency number.

Preserve the evidence

Use a fixed random seed for synthetic noise and also test deterministic boundary traces. Report raw and filtered commands, not only a smoothed plot. The pass condition must include resting stability, a step response, endpoint clipping and dead-zone behavior. A filter that never changes its output has perfect resting stability and is a completely unusable controller.

Practical checkpoint

Lab 31.4: Noisy joystick, ADC and latency pipeline. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R30: mixed-signal and measurement concepts; R10: sampling and filtering.

Core quiz - original program questions / 6 points

31.4-C1. How many distinct codes does a 12-bit converter provide?

31.4-C2. Does 12-bit resolution guarantee 12-bit accuracy?

31.4-C3. Why filter before sampling to prevent aliasing?

Transfer check - new application questions / 4 points

31.4-T1. For alpha=0.25, how many discrete updates are needed to reach at least 90% of a unit step from zero?

31.4-T2. At fs=1 kHz, why can digital processing alone not identify whether an observed 300 Hz sampled cosine came from 300 or 700 Hz?

Do not open the key until you have recorded an answer and its reasoning.

Module 32 - Digital logic, state and timing

Prerequisites: Modules 06, 15-16 and 31.

Module project: Build and test an arithmetic unit, a finite-state controller and a synchronous queue model. For every block, provide a truth table or transition specification before coding its implementation.

32.1 - Boolean logic and combinational building blocks. Translate a logical specification into a circuit with no stored state.

32.2 - Registers, finite-state machines and synchronous timing. Model state transitions at clock boundaries.

32.3 - Binary arithmetic, fixed point and overflow policies. Choose representations that preserve the needed range and precision.

32.4 - Clock domains, reset and metastability. Recognize where synchronous assumptions stop applying.

Lesson 32.1 - Boolean logic and combinational building blocks

Learning outcome: Translate a logical specification into a circuit with no stored state.

Game-engineering connection: A collision mask, address decoder and arithmetic unit all reduce to logic functions. Specify the function first, then choose a circuit organization that implements it.

Separate function from implementation

Combinational logic computes an output from current inputs without intentional stored state. A truth table gives the complete function for a small number of input bits. A Boolean expression, a network of gates and a software model can describe the same function with different implementation costs. A sequential circuit additionally depends on stored state. An accidental latch converts what you thought was a combinational function into a history-dependent system. [R15, R31]

For two inputs A and B, AND is true only when both are true; OR when at least one is true; XOR when they differ. NOT complements one bit. De Morgan's laws relate a complemented AND to an OR of complements and vice versa. These identities are useful for proving equivalence, but a shorter expression on paper need not map to a shorter physical path after technology mapping and routing.

A 2-to-1 multiplexer selects A when S=0 and B when S=1. Its single-bit equation is Y=(NOT S AND A) OR (S AND B). A decoder maps an encoded input to a selected output line. With enable E, a 2-to-4 decoder has one asserted output when E=1 and none when E=0 under the specified active-high convention. Undefined input encodings need a policy, not a hidden assumption.

Derive the full adder

A one-bit full adder receives A, B and carry-in C. The sum bit is A XOR B XOR C. The carry-out is AB + AC + BC, where multiplication means AND and addition means OR. Equivalently, define generate G=AB and propagate P=A XOR B, then carry-out=G OR (P AND C). Exhaust all eight input combinations before composing wider adders.

A W-bit ripple adder connects each carry to the next stage. It is conceptually simple, but the worst carry path can span many bits. A carry-lookahead or prefix structure reorganizes carry computation, trading logic and wiring for path depth. At this stage, do not claim a timing improvement without either a named analytical delay model or an implementation report.

Width is part of the specification

Adding two four-bit unsigned values produces a mathematical result from 0 through 30. Four result bits can hold only 0 through 15. Specify whether the result includes a fifth carry bit, discards it for modular arithmetic, or saturates. “It is an adder” does not resolve that question.

Signedness is an interpretation of bit patterns. The same four-bit pattern 1111 means 15 unsigned or -1 in two's complement. Combinational gates do not know which interpretation you intended; interpretation changes extension, comparison, overflow detection and meaning at the interface. Use explicit conversions in the reference model rather than letting a host language choose accidentally.

Worked example: prove a four-bit adder

With A=1101 (13) and B=0111 (7), the full five-bit result is 10100 (20). The low four bits are 0100 and carry-out is 1. If those inputs are interpreted as signed four-bit values, they are -3 and +7, whose sum +4 is representable. Carry-out is therefore not a universal signed-overflow flag.

For exhaustive verification, iterate A from 0 to 15, B from 0 to 15 and carry-in from 0 to 1: 16 times 16 times 2 = 512 cases. For each, compare a gate-derived ripple model with independent integer arithmetic. Check both result bits and carry-out. Reusing the same carry recurrence in both the implementation and reference can preserve the same bug in both.

For the decoder, exhaust all four select values and both enable values. For each enabled case, assert exactly one output bit is high and that its position equals the select. The bit-count assertion alone is insufficient: a decoder that always chooses output zero has one high bit but the wrong function.

Hazards and structural checks

A logic function can be correct after settling yet glitch during input transitions because paths have different delays. Pure zero-delay truth-table simulation does not establish glitch-free behavior. Register a control signal or use an appropriate implementation and timing discipline when a transient could trigger a side effect. Avoid combinational feedback in beginner synchronous work unless the storage behavior is intentional and analyzed.

The engineering habit is to separate three claims: Boolean equivalence, timing behavior and physical implementation suitability. An exhaustive truth-table test supports the first for the tested finite domain. It does not establish all three. This distinction returns throughout FIFO, processor and pixel-pipeline design.

Practical checkpoint

Lab 32.1: Exhaustive combinational building blocks. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: combinational logic; R31: Boolean logic and arithmetic projects.

Core quiz - original program questions / 6 points

32.1-C1. What distinguishes combinational from sequential logic?

32.1-C2. What does a two-to-one multiplexer do?

32.1-C3. Does the shortest Boolean expression always imply the fastest circuit?

Transfer check - new application questions / 4 points

32.1-T1. A four-bit adder receives unsigned 13 and 7. Give the low result, carry-out and number of exhaustive input cases including carry-in.

32.1-T2. Why does a one-hot assertion not fully verify a 2-to-4 decoder?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 32.2 - Registers, finite-state machines and synchronous timing

Learning outcome: Model state transitions at clock boundaries.

Game-engineering connection: A game-facing hardware block must remember whether it is idle, processing or finished. State machines make those transitions reviewable before the implementation becomes complicated.

The edge separates present state from next state

A synchronous state machine has a current-state vector Q, an input vector X, a transition function F and an output function. At the active clock edge, registers capture F(Q,X) after the inputs and combinational result meet timing requirements. Between edges, Q remains stable in the ideal model. Drawing the transition table before writing RTL exposes missing priorities and impossible expectations. [R15]

A Moore-style output depends on state. A Mealy-style output may also depend on current inputs. That can reduce response latency but introduces a combinational input-to-output path whose timing and glitch behavior must be reviewed. Neither style is universally better. State what the receiver samples and on which edge.

Design an explicit controller contract

Consider IDLE, RUN and DONE. Reset has highest priority and returns to IDLE. Abort has next priority and returns to IDLE from any state. In IDLE, start enters RUN. In RUN, complete enters DONE. In DONE, acknowledge returns to IDLE. All other cases hold state. Under this contract, start while RUN is ignored, and a same-cycle acknowledge and start while DONE returns to IDLE without launching a new operation.

Those priorities are choices, not laws of hardware. Write tests for conflicting inputs so later refactoring cannot change the choices silently. Define an illegal-state recovery policy. A default recovery path is useful, but by itself it is not a guarantee against all physical single-event or clock faults.

Setup and hold are different inequalities

For a simple register-to-register path with zero skew, setup requires Tclk >= tCQ,max + tlogic,max + tsetup + Usetup. Here tCQ is the launch register's clock-to-output delay and U represents the uncertainty budget. Setup concerns the latest arrival before the next intended capture edge. A positive slack is the available time minus required path delay.

Hold constrains the earliest data change relative to the same capture edge. In a simplified zero-skew model, tCQ,min + tlogic,min >= thold + Uhold. It is a minimum-delay check, so slowing the clock does not generally repair it. Physical clock skew has different signs in setup and hold equations; derive the arrival times when using a nonzero-skew worksheet rather than adding the same margin everywhere.

Changing a long combinational path into two stages can improve the achievable clock period while adding latency. The interface must then align control and data through both stages. A performance claim should distinguish latency in cycles, latency in time and steady-state throughput.

Worked timing and state trace

Assume tCQ,max=0.8 ns, maximum logic delay=5.6 ns, setup=0.6 ns and setup uncertainty=0.5 ns. The simplified minimum period is 7.5 ns, corresponding to about 133.33 MHz. At an 8 ns period, setup slack is 0.5 ns. These are worksheet values, not a device timing signoff.

For hold, let tCQ,min=0.1 ns, minimum logic=0.05 ns, hold=0.2 ns and hold uncertainty=0.05 ns. Earliest data arrival is 0.15 ns while the requirement is 0.25 ns: slack is -0.10 ns. Doubling the period leaves that inequality unchanged. A real implementation tool may add delay or adjust clock/data routing under target-specific rules.

Starting in IDLE, inputs start, none, complete, acknowledge on four successive edges give post-edge states RUN, RUN, DONE, IDLE. Starting in RUN with complete and abort both asserted gives IDLE because abort has higher priority. A trace should name whether a value is before or after the edge; many apparent “off-by-one” disagreements are undocumented observation phases.

Clock enable is not an improvised clock

To update state less often, use a clock enable in a single clock domain for these exercises. A combinationally gated clock can glitch or violate distribution assumptions. Dedicated clock-control resources and their constraints belong to a later, target-specific implementation review. Likewise, a slow signal used as a clock creates another domain, even when its frequency looks harmless.

For the module project, maintain separate truth/transition specifications for the arithmetic unit, controller and queue. Their integration should share a documented reset and edge convention. Predict one normal trace and one conflicting-input trace before running the simulator.

Practical checkpoint

Lab 32.2: Controller transition and timing workbook. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: sequential logic and finite-state machines; R31: sequential logic.

Core quiz - original program questions / 6 points

32.2-C1. What does a register capture in a synchronous model?

32.2-C2. What simplified maximum frequency follows from an 8 ns period?

32.2-C3. Does slowing the clock automatically fix hold violations?

Transfer check - new application questions / 4 points

32.2-T1. Given maximum delays 0.8+5.6+0.6 ns plus 0.5 ns uncertainty, what period is required and what is setup slack at 8 ns?

32.2-T2. A path has 0.15 ns earliest data arrival and a 0.25 ns hold requirement. What is its hold slack, and does changing period from 8 to 16 ns fix it?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 32.3 - Binary arithmetic, fixed point and overflow policies

Learning outcome: Choose representations that preserve the needed range and precision.

Game-engineering connection: A fixed-point vector unit can save hardware, but only when its numerical contract is precise enough for the game system that uses it.

Bits need a numerical contract

An unsigned W-bit value ranges from 0 to 2^W-1. A two's-complement signed W-bit value ranges from -2^(W-1) to 2^(W-1)-1. Sign extension repeats the sign bit; zero extension inserts zeros. Choosing the wrong extension changes a negative operand before any arithmetic takes place. [R15, R31]

Wrapping addition keeps the low W bits, implementing arithmetic modulo 2^W. Saturating addition clamps to the representable endpoints. Neither policy is universally correct: a counter may deliberately wrap, while a color or bounded force may require saturation. A saturating intermediate result also changes associativity, so regrouping arithmetic can change an algorithm's output.

For signed addition, overflow occurs when equal-sign operands produce an opposite-sign result after truncation. Carry-out alone is not that condition. An easier reference implementation adds in a wider mathematical integer, then explicitly applies wrapping or clipping. Keep that reference structurally independent from the RTL overflow detector.

Define fixed point without ambiguous shorthand

This volume describes a format by total width W, signedness and fractional-bit count F. The stored integer q represents q/2^F. For signed W=16 and F=8, the range is -128 through 127.99609375, with step 1/256=0.00390625. The sign bit is included in the total 16 bits. Different uses of names such as Q8.8 can count integer or sign bits differently; the explicit tuple avoids that ambiguity.

Adding values with the same F requires no rescaling, but may overflow. Multiplying two W-bit stored integers produces a value with 2F fractional bits and up to 2W bits of signed precision. To return to F fractional bits, divide the widened product by 2^F with a named rounding rule, then apply the overflow policy. Truncating operands before multiplication is not equivalent.

Rounding negative values deserves its own test

Truncation toward zero, floor and round-to-nearest-even differ. An arithmetic right shift of a negative two's-complement integer usually implements a floor-like power-of-two division in the intended hardware arithmetic, not truncation toward zero. Do not assume the host language and HDL use the same default for every operator.

For round-to-nearest-even, compare the discarded magnitude with half the divisor. A tie selects an even retained integer. The rule avoids a consistent tie direction under appropriate data distributions, but it is not a guarantee of zero application-level bias. Include positive and negative halfway cases in directed tests. Report clipping separately from rounding error because saturation can dominate the numerical loss.

Worked arithmetic examples

In signed eight-bit arithmetic, 120+20=140 mathematically. Wrapping gives bit pattern 10001100, interpreted as -116. Saturating gives 127. For -120 + -20 = -140, wrapping gives +116 and saturating gives -128.

In signed W=16,F=8 format, 1.5 is stored as 384 and -0.75 as -192. The full product is -73,728 with 16 fractional bits. Rescaling by 256 yields -288 exactly, representing -1.125. No rounding error occurs because this result is exactly representable.

For a small tie example, use F=2 and stored product +6: dividing by 4 gives +1.5 stored units. Nearest-even returns +2. Product +10 gives +2.5 and also returns +2. The negative cases -6/4 and -10/4 return -2. A floor rule would return -3 for -10/4, demonstrating why a test containing only positive products is incomplete.

Quantify error where the workload uses it

For a nonclipped nearest-rounded value, representation error is at most half one output step under the specified conversion. A chain of operations can accumulate or amplify error, and rounding at each stage can differ from rounding once at the end. In a dot product, widen the accumulation if the expected range requires it, then document when reduction occurs.

For a game subsystem, test the meaningful distribution as well as mathematical corners: small velocities near rest, values near collision thresholds, negative coordinates and repeated accumulation. Compare against an independent high-precision calculation and report maximum error, mean signed error and clipping count. Numerical quality and performance must be reported together; a fast implementation that changes the simulation unacceptably has not met the contract.

Practical checkpoint

Lab 32.3: Bit-accurate arithmetic and error report. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: arithmetic; R31: ALU construction; R35: integer arithmetic semantics as a software/hardware contract.

Core quiz - original program questions / 6 points

32.3-C1. What is the signed 8-bit two's-complement range?

32.3-C2. What does stored value 24 mean with scale 1/16?

32.3-C3. Why use a wider intermediate for multiplication?

Transfer check - new application questions / 4 points

32.3-T1. Give wrapped and saturated signed eight-bit results for 120+20, then state the range of signed W=16,F=8.

32.3-T2. With nearest-even division by four, what do products +6, +10, -6 and -10 become?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 32.4 - Clock domains, reset and metastability

Learning outcome: Recognize where synchronous assumptions stop applying.

Game-engineering connection: A button, audio source or separate pixel clock can violate the timing assumptions of the main logic even when the signal changes very slowly.

A clock domain is a timing relationship

Synchronous logic assumes that data reaches a register within its setup and hold window. An asynchronous signal can change arbitrarily close to the sampling edge and leave the register temporarily metastable. The register may take longer than normal to resolve to a valid digital level. A digital functional simulator does not reproduce the analog probability distribution of that behavior. [R15, V7-R01]

A chain of destination-clocked synchronizer registers gives an initial metastable event additional resolution time before the signal is used. It reduces propagation risk; it does not make the probability mathematically zero. The appropriate number of stages, placement, constraints and reliability calculation depend on the target and event rates. Do not fan out the first-stage output to functional logic.

Choose a protocol for the information being transferred

A slowly changing single-bit level can often use a synchronizer under a stated rate assumption. A narrow source pulse may occur entirely between destination edges and be missed. A toggle protocol converts an event into a state change, but it still needs an event-rate bound or acknowledgement so two toggles cannot cancel before observation.

A multi-bit word cannot generally be made coherent by placing each bit through an independent synchronizer. Different bits can be observed at different times. A binary transition 0111 to 1000 changes four bits; a receiver can temporarily observe a word that the source never intended. Gray coding limits changes between adjacent encoded states but is not a universal bus-transfer solution. Physical skew constraints and the intended use of the code still matter. [V7-R01]

For occasional words, use a handshake that establishes ownership. One simple bundled-data protocol has the source write a holding register, keep the word stable, and toggle a request bit. The destination synchronizes the request, captures the stable data under a reviewed settling assumption, then returns an acknowledgement toggle through the source synchronizer. The source does not alter the held word until it observes acknowledgement. This is not permission to ignore physical timing of the data path.

Reset is part of the protocol

The request and acknowledgement toggles need a consistent initial relationship. Resetting one side alone can make an old request look new or erase ownership information. State whether both sides reset together, whether in-flight work is dropped, and how independent resets are recovered. For a physical implementation, reset assertion/deassertion and recovery/removal constraints need a domain-aware plan; asynchronous release is not automatically safe.

For most other labs, keep one clock and a synchronous active-high reset to minimize hidden assumptions. The CDC lesson is a protocol exercise, not a requirement to add extra clocks to the capstone. A clock enable often avoids an unnecessary domain, but an external interface may still require a crossing.

Worked protocol trace

Suppose source and destination clocks have unrelated 10 ns and 14 ns periods. Initially req=ack=0. The source holds word 0xA5 and toggles req to 1. The destination's two-stage request synchronizer observes that change after destination edges whose exact count depends on phase. When synchronized req differs from its recorded acknowledgement state, it captures the data and updates ack to the observed request value. The returning acknowledgement eventually matches req at the source, which may then release the held word.

If the source overwrites 0xA5 with 0x5A before acknowledgement, a fast request path and slow data path can cause capture of a mixed or unintended word. A functional protocol test can detect that the source violated stable ownership. It cannot derive an analog metastability mean time between failures from a clean waveform.

Verify the contract, not an impossible guarantee

Sweep source/destination phase offsets and frequency ratios. Randomly delay observation of request and acknowledgement by allowed whole-cycle amounts to test whether the protocol depends on an exact latency. Assert that the source payload remains stable while outstanding and that each accepted request yields at most one destination acceptance. With a fairness assumption that both clocks continue and acknowledgements eventually propagate, you may also check eventual completion.

Name those assumptions. “Every word arrives” is not true if the destination clock stops forever or reset discards pending work. Keep safety claims, such as no double delivery, separate from progress claims, such as eventual delivery. An asynchronous FIFO, which handles a sustained stream across domains, is a later extension; do not substitute an unreviewed dual-clock pointer trick for this bounded exercise.

Practical checkpoint

Lab 32.4: Bundled-data ownership crossing model. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: synchronization and asynchronous inputs; deepen with R50 in later verification lessons.

Core quiz - original program questions / 6 points

32.4-C1. Does a two-register synchronizer eliminate all metastability risk?

32.4-C2. Why not independently synchronize every bit of a changing binary bus?

32.4-C3. Can ordinary digital simulation prove an analog metastability failure rate?

Transfer check - new application questions / 4 points

32.4-T1. Why does a two-register synchronizer on a one-cycle pulse not guarantee that a slower destination sees the pulse?

32.4-T2. In a bundled-data handshake, when may the source change the payload, and what additional physical assumption is still required?

Do not open the key until you have recorded an answer and its reasoning.

Module 33 - SystemVerilog and verification-driven RTL design

Prerequisites: Module 32; Python and C++ foundations complete.

Module project: Implement a parameterized synchronous FIFO in RTL with a Python or C++ reference model, automated tests, assertions and synthesis reports. Keep testbench behavior separate from synthesizable hardware.

33.1 - RTL semantics and synthesizable SystemVerilog. Write a hardware description rather than a sequential software imitation.

33.2 - Testbenches, scoreboards and constrained stimulus. Check behavior against an independent specification.

33.3 - Assertions, coverage and formal reasoning. Distinguish observed tests from properties proven under assumptions.

33.4 - Synthesis, constraints and FPGA resource mapping. Connect verified behavior to a realizable implementation.

Lesson 33.1 - RTL semantics and synthesizable SystemVerilog

Learning outcome: Write a hardware description rather than a sequential software imitation.

Game-engineering connection: SystemVerilog looks like a programming language, but synthesizable RTL describes concurrent state and dataflow. The important question is what circuit a statement implies.

Concurrent blocks, edge-triggered state

A module contains concurrent processes and continuous connections. In a clocked process, nonblocking assignments evaluate their right-hand sides using values visible when the process executes and schedule updates for the nonblocking-update region. That lets multiple registers model simultaneous edge-triggered state changes. Procedural statement order still matters when assigning the same variable more than once; avoid accidental multiple writers. [R32, R33]

Use a single-writer discipline. A register should have one well-defined clocked owner. Use an always_ff block for intended edge-triggered storage and always_comb for intended combinational logic where supported by the chosen tools. Tool support is version-specific; the package uses a deliberately small SystemVerilog subset and records actual execution separately from reference code availability.

Trace nonblocking assignments by hand

Consider registers a=1 and b=2 before an edge, followed by a<=b and b<=a in one clocked block. Right-hand sides use the old values, so after the edge a=2 and b=1. If both assignments use blocking equals in the same sequential process, a first becomes 2 and b then reads that changed a, leaving both 2. The latter is not the intended two-register swap.

Similarly, q<=d followed by r<=q gives r the previous q, not the new d. This is a two-stage pipeline. A testbench that observes r too early in the simulator's event sequence may still see the preceding state. Define an observation phase after scheduled state updates and combinational settling rather than relying on a race at the rising edge. [R34]

Combinational logic needs a value on every path

A combinational process should assign every output for all input cases. A common pattern assigns defaults first, then overrides them in selected branches. Omitting an assignment can imply a latch because the previous value must be retained. An intentional latch is a distinct design choice; do not silently create one while intending pure combinational logic.

An RTL loop with static bounds may describe replicated hardware. It is not automatically a software loop consuming one iteration per clock. Conversely, a clocked accumulator updated once per edge is sequential hardware even when its arithmetic expression is short. Synthesis elaborates parameters and maps the resulting structure to available resources. Simulation-only file I/O, arbitrary delays and dynamic software structures are not generally portable synthesizable hardware.

Width and reset decisions affect real resources

Make arithmetic widths explicit. Unsized literals and mixed signed/unsigned expressions can introduce extensions or truncations different from the intended mathematics. The reference FIFO gives occupancy enough bits to represent every integer from zero through DEPTH, not only zero through DEPTH-1. A depth-one FIFO still requires valid nonzero declaration widths.

Reset control state when required; do not automatically reset every memory bit merely to remove unknowns from a waveform. A FIFO can reset its occupancy and pointers while leaving storage contents unspecified, provided it never exposes them as valid before a write. Resetting a large array can prevent the desired memory inference on some targets. The output-data contract must say whether data is meaningful when valid is low.

Worked RTL: a saturating counter

For an unsigned W-bit counter with synchronous reset and enable, define reset > enable, and hold at all ones rather than wrap. The next-state equation is zero on reset; otherwise q+1 when enable is true and q is not maximum; otherwise q. A combinational “done” may be q==MAX. If done should pulse only on entering the maximum state, it needs a transition-based definition or additional state.

The following pattern describes the state update; it is not a complete module:

```
always_ff @(posedge clk) begin
    if (rst) q <= '0;
    else if (enable && q != {W{1'b1}}) q <= q + 1'b1;
end
```

The missing final else in a clocked block means hold state, which is intended. The same omission in a combinational block could imply storage. Context changes the interpretation. With W=3, q=6 and enable high, successive post-edge values are 7,7,7; reset then produces 0 even if enable remains high.

Do not confuse passing simulation with synthesis

Lint checks structural and semantic issues before long runs. Synthesis checks whether the selected subset can be implemented and reports what resources were inferred. Neither replaces a testbench. A design can synthesize cleanly while computing the wrong result, or simulate correctly while relying on a nonsynthesizable construct. Preserve source, tool version, command and warnings as different evidence from output waveforms.

Practical checkpoint

Lab 33.1: Counter, swap and scheduling semantics. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R32: language support, warnings and simulation; R33: synthesis concepts and supported constructs.

Core quiz - original program questions / 6 points

33.1-C1. Why assign every combinational output on every path?

33.1-C2. What does the nonblocking swap example do?

33.1-C3. Does a source-code loop necessarily take multiple hardware clock cycles?

Transfer check - new application questions / 4 points

33.1-T1. Before an edge a=1 and b=2. What follows a<=b; b<=a, and why?

33.1-T2. Why is an omitted else sometimes intentional in always_ff but dangerous in intended combinational logic?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 33.2 - Testbenches, scoreboards and constrained stimulus

Learning outcome: Check behavior against an independent specification.

Game-engineering connection: A FIFO can pass thousands of random cycles while still failing the first time it becomes full. Verification begins with an explicit acceptance contract and a coverage plan.

Define the transfer event first

This package's synchronous FIFO uses input-valid/input-ready and output-valid/output-ready. A transfer occurs only at a rising edge when the corresponding valid and ready are both true and reset is inactive. Output data is meaningful only while output-valid is true. The producer holds valid and payload stable while blocked. The consumer can apply backpressure by deasserting ready.

The reference policy is conservative: input-ready means occupancy<DEPTH, independent of a same-cycle read. Output-valid means occupancy>0. At full occupancy, a simultaneous read request and write request accepts only the read, even though space will exist afterward. At empty occupancy, simultaneous requests accept only the write; there is no combinational bypass. These are deliberate policies, not universal FIFO behavior.

The transition is count_next=count+push-pop, with push and pop computed from pre-edge flags. If both occur in an interior state, occupancy stays the same, the old head is removed and a new tail is inserted. A replacement-on-full FIFO can be a valid extension, but it must change the reference contract and boundary tests, not just one line of RTL.

The scoreboard must be independent

Use a software deque or a simple abstract sequence as the reference. On an accepted read, compare output data to the pre-edge sequence head. On an accepted write, append the input after accounting for the read. On reset, clear expected ownership according to the flush policy. Do not compute expected data by copying the DUT's pointer arithmetic; that can duplicate the same bug.

Separate a driver, monitor and scoreboard conceptually even in a small testbench. The driver supplies legal input behavior. The monitor identifies actual accepted transfers. The scoreboard compares those transfers with the specification. A producer withdrawing a blocked transaction is a driver violation unless a cancellation protocol is explicitly defined.

Stimulus needs a plan

Directed tests establish empty, full, depth-one, pointer wrap, simultaneous operations, reset while nonempty, output stall and recovery. Randomized tests explore combinations that were not anticipated. Use reproducible seeds and record parameters and input events. A test count is not the same as coverage: 10,000 cycles that stay almost empty may never exercise the full boundary.

Functional coverage should count meaningful events and transitions, such as empty-to-nonempty, full-to-not-full, interior simultaneous transfer, wrap with data preservation and reset with pending work. Test a non-power-of-two depth such as three; automatic binary pointer overflow is wrong for that configuration unless the design handles it explicitly.

Testbench scheduling is part of correctness

Drive before the sampling edge, sample handshake values consistently, then check post-edge state after nonblocking updates settle. In a cocotb bench, a rising-edge trigger by itself need not mean every downstream HDL process has completed. The documented timing model distinguishes value-change, read/write and read-only phases. Follow the installed-version documentation rather than inserting unexplained tiny delays everywhere. [R34]

The supplied SystemVerilog bench drives at a falling edge, records pre-edge interface values, advances through the rising edge, and checks after a small declared simulation step. That step is a testbench observation convention, not a physical circuit delay. Keep that distinction in the report.

Worked boundary trace

With DEPTH=2 and queue [A,B], input-valid=1 carrying C and output-ready=1, pre-edge input-ready is 0 and output-valid is 1. The accepted output is A, the old head. The post-edge queue is [B]. C remains the producer's outstanding word because it was not accepted.

On the next edge, with the same offered C and output-ready=0, input-ready=1 and only the write is accepted, producing [B,C]. On a later interior simultaneous operation with [B] and offered D, the consumer receives B and the new queue becomes [D]. Writing the sequence explicitly prevents accidental new-data/old-data confusion.

Prove that the checker can fail

Mutation testing deliberately changes a small behavior, such as returning the newest element instead of the oldest, skipping a pointer wrap or accepting a write when full. A useful test suite catches the mutation and reports the first divergence with enough state and seed information to reproduce it. A suite that always reports success because assertions were disabled is worse than no suite: it creates false confidence.

Practical checkpoint

Lab 33.2: FIFO scoreboard and reproducible failure. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R34: writing testbenches, triggers and simulator timing; R32: simulation and tracing.

Core quiz - original program questions / 6 points

33.2-C1. Should a scoreboard count attempted or accepted transactions?

33.2-C2. Why save random seeds?

33.2-C3. Why deliberately inject a known bug?

Transfer check - new application questions / 4 points

33.2-T1. Under this FIFO policy, depth two is full with [A,B], input offers C and output-ready is high. Which transfers occur and what remains?

33.2-T2. Why should the scoreboard use an abstract sequence instead of copying the RTL read/write-pointer equations?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 33.3 - Assertions, coverage and formal reasoning

Learning outcome: Distinguish observed tests from properties proven under assumptions.

Game-engineering connection: Tests show what happened on selected traces. Assertions and formal reasoning let you express what must never happen, and which assumptions make that statement meaningful.

Three different kinds of evidence

Simulation explores chosen input traces. Bounded formal checking explores all allowed traces up to a specified depth in a formal model. An unbounded safety proof, when the selected method succeeds, establishes the property for all reachable modeled states under the assumptions. None of these automatically proves the analog timing or physical reliability of a circuit. Report the exact property, model, assumptions, engine and result. [R50]

An assertion is a condition the design must satisfy. An assumption constrains the environment or initial state. A cover asks whether an interesting behavior is reachable. Confusing them can make a broken design appear correct: assuming the FIFO is never full removes the very case a full-handling assertion is meant to check.

Write invariants from the specification

For a FIFO, occupancy must remain between zero and DEPTH. Push and pop occur only on accepted handshakes. If reset flushes the queue, the next meaningful state is empty. While output-valid is true and output-ready is false, output-valid and payload must remain stable until either transfer or reset under the declared interface contract.

Occupancy bounds are necessary but do not prove ordering. A FIFO that outputs every item twice while dropping alternate items could keep the count in range. Ordering requires a stronger sequence property, a tracked symbolic item or an independent reference queue. State exactly which properties the supplied harness contains and which the learner still needs to add.

Understand temporal observation

A property about the next edge compares sampled state across time. A past-value reference is undefined until a previous sampling event exists, so guard initial history. Reset may disable a property or create a specific expected transition; those choices are not interchangeable. The simple formal starter uses past-valid guarding and an initial reset assumption, then allows later resets to test the flush rule.

A legal ready/valid source holds its data and valid while blocked. This can be an environment assumption for verifying a receiving component, but the matching behavior must be asserted when verifying the producer. Contract composition depends on using assumptions and guarantees on the correct side of each interface.

Safety, progress and vacuity

“No accepted word is corrupted” is a safety idea: a finite bad trace demonstrates failure. “Every accepted word is eventually consumed” is a progress statement and is false if the consumer keeps ready low forever. A liveness proof needs a justified fairness or bounded-service assumption, or a different claim that does not promise completion.

A proof can be vacuous when the antecedent of an implication never occurs or the assumptions exclude useful activity. Add cover goals for reaching full occupancy, draining to empty after activity and completing simultaneous read/write in an interior state where the selected depth permits it. A depth-one conservative FIFO cannot perform an interior simultaneous transfer; an unreachable cover there is a policy fact, not automatically a tool bug.

Worked reasoning: stability under backpressure

Suppose the current queue head is H, output-valid=1 and output-ready=0. There is no accepted pop. With no reset, a legal FIFO cannot change the head. A write may append at the tail when space exists, but it cannot replace H. Therefore the next sampled output remains valid with data H. This argument depends on the reset exclusion and the no-bypass queue definition.

Now consider count=DEPTH. Since input-ready=0, push=0. A pop gives count_next=DEPTH-1; no pop keeps it full. Count cannot exceed DEPTH from this state. For an interior state, push and pop change count by at most one in the allowed directions. An inductive proof combines a reset base case with preservation across all transition cases. That establishes the bounds invariant for the model, but still says nothing about which payload value is stored.

Formal workflow without inflated claims

Begin with the smallest width and depth that preserve the behavior of interest. Inspect a failing counterexample as a waveform and a sequence of accepted transactions. If the result is unknown or times out, report unknown; do not treat absence of a counterexample as a proof. A bounded pass at depth 20 is a depth-20 bounded result, not an unbounded certificate.

The bundled formal file is an exercise starter for occupancy and stability checks. It does not arrive with a proof certificate, and this package does not claim formal execution where no formal tool was available. The learner must run, inspect, extend and record the harness as part of the lesson. An honest open evidence item is preferable to a fabricated green check.

Practical checkpoint

Lab 33.3: Properties, covers and assumption audit. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R50: getting started, FIFO example, safety, cover and liveness; R34: assertion-oriented simulation checks.

Core quiz - original program questions / 6 points

33.3-C1. Does reaching 100% of chosen coverage bins prove correctness?

33.3-C2. Why can overconstrained assumptions hide a bug?

33.3-C3. What limitation must accompany a bounded proof?

Transfer check - new application questions / 4 points

33.3-T1. Does proving 0<=count<=DEPTH establish FIFO ordering?

33.3-T2. A bounded check finds no counterexample through 20 cycles. What claim is justified, and what must not be claimed?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 33.4 - Synthesis, constraints and FPGA resource mapping

Learning outcome: Connect verified behavior to a realizable implementation.

Game-engineering connection: A correct RTL description is only one stage on the way to hardware. Synthesis tells you what structure was inferred; implementation and timing analysis answer different questions.

Follow the transformation chain

Elaboration resolves parameters, generates and module hierarchy. Synthesis converts the supported RTL behavior into an intermediate netlist, optimizes it and maps it toward logic and memory resources. Technology mapping targets a particular cell or FPGA-resource library. Placement and routing add physical locations and interconnect. Static timing analysis evaluates constrained paths using a target timing model. [R33]

A generic Yosys synthesis report is useful evidence that the selected RTL can be reduced to a logic network and that resource structure is understood. It is not a routed FPGA result and does not supply a valid maximum clock frequency by itself. A file named “constraints” is not evidence that a tool read and applied it. Record the exact command and inspect logs.

Memory style changes the architecture

A small FIFO can use registers with multiplexers, distributed memory or block memory depending on the target, size, ports, read behavior and coding style. The reference FIFO uses an asynchronous head read for pedagogical simplicity and a conservative handshake. That choice may map differently from a registered-read block-memory FIFO. Do not promise automatic block-RAM inference from an unpacked array declaration.

A registered-read implementation can add latency or require an output staging register while preserving the external transfer contract. Define first-word visibility, read-during-write behavior, reset handling and output stall behavior. Compare equivalent external functionality, not merely the same-looking source arrays.

Resetting every data word, adding unsupported read/write port combinations or demanding a particular simultaneous read/write result can change inference. These are architectural constraints. Inspect what was actually inferred before explaining a resource difference as an optimizer failure.

Constraints encode environment assumptions

A primary clock constraint defines a period and waveform for a known port. Input and output delays describe timing relationships to external logic. Generated clocks and asynchronous crossings need their own treatment. A false-path exception is a claim that a path need not satisfy a particular timing relationship; it does not insert a synchronizer or make an unsafe crossing safe. [V7-R01]

This volume includes a commented target-constraint worksheet rather than a universal board file. There is no chosen FPGA pinout, I/O voltage or routed timing corner. The core requirement can be satisfied with identified generic synthesis evidence plus a clear statement that target implementation timing remains unmeasured. A physical extension requires the actual device, board schematic and supported tool flow.

Worked comparison: depth and read policy

For W=16, a depth-three FIFO contains 48 payload storage bits at the abstract level; depth eight contains 128. That arithmetic does not predict LUT count, block-RAM usage or routing delay. Control includes pointers and occupancy. Occupancy needs ceil(log2(DEPTH+1)) bits, so depth eight needs four count bits to represent eight, while a pointer into eight entries needs three.

Compare generic synthesis of depth three and depth eight with the same data width and policy. Report tool version, top module, parameter overrides, inferred memories before mapping and cell counts after mapping. If one report maps storage to flip-flops, explain the selected generic flow rather than relabeling those cells as block RAM.

A second comparison may replace the asynchronous read with a registered-read architecture. Predict which interface latency changes before running tests. Rerun the behavioral suite against the new external contract or a compatibility wrapper. An apparent resource improvement is not valid if the altered design silently drops the output under backpressure.

Timing reports need complete context

A setup slack of +0.2 ns means something only with the required period, path endpoints, clock definitions, uncertainty, corner and analysis stage. A design with unconstrained paths can show favorable slack on the paths that happened to be constrained. Likewise, post-synthesis estimates omit routed interconnect effects that may dominate a physical target.

For the worksheet, reuse the setup and hold derivations from 32.2. For actual implementation, inspect unconstrained-path reports and distinguish setup from hold failures. Do not subtract generic gate counts from a frequency target to invent Fmax. Power claims also require a target model, activity assumptions and methodology; toggling a simulation signal is not a watt measurement.

Close the module with reproducibility

The deliverable is a verified parameterized FIFO plus its specification, reference, tests, properties, reset policy and synthesis evidence. Keep commands reproducible from a clean directory. Record skipped stages as not run. Synthesis is a learner requirement even though a runnable script is supplied; shipping the script is not the same as executing it.

Practical checkpoint

Lab 33.4: FIFO synthesis and mapping comparison. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R33: synthesis and memory mapping; R32: lint; selected FPGA tool documentation only after a target is chosen.

Core quiz - original program questions / 6 points

33.4-C1. Does successful synthesis establish timing closure?

33.4-C2. Why inspect inferred memory resources?

33.4-C3. Why use identical constraints when comparing designs?

Transfer check - new application questions / 4 points

33.4-T1. At width 16 and depth eight, how many abstract payload bits are present, and how wide must occupancy be?

33.4-T2. Why cannot a generic synthesis cell count establish routed FPGA Fmax?

Do not open the key until you have recorded an answer and its reasoning.

Module 34 - Designing a small processor

Prerequisites: Modules 16 and 32-33. Implement a documented subset, not an unqualified full-ISA claim.

Module project: Create a simple processor model and RTL implementation for a clearly specified instruction subset. Run the same short programs on an instruction-level reference and the RTL, comparing architectural state at retirement.

34.1 - Instruction sets, datapaths and single-cycle execution. Separate programmer-visible behavior from its implementation.

34.2 - Pipelining, forwarding, stalls and branch recovery. Preserve architectural behavior while overlapping instructions.

34.3 - Memory interfaces, caches and protection boundaries. Design memory behavior as a protocol rather than an instantaneous array.

34.4 - Processor verification, toolchains and architectural evidence. Demonstrate instruction correctness with reproducible programs and traces.

Lesson 34.1 - Instruction sets, datapaths and single-cycle execution

Learning outcome: Separate programmer-visible behavior from its implementation.

Game-engineering connection: Building a small processor makes instruction execution, register dependencies and memory behavior concrete. The goal is a precise machine, not a miniature marketing claim about a commercial CPU.

ISA and microarchitecture are different contracts

An instruction-set architecture specifies what software can observe: instruction encodings, registers, arithmetic rules, memory behavior and supported faults. A microarchitecture chooses how to produce that behavior with datapaths and control. The same ISA can be implemented by a single-cycle machine, a multicycle controller or a pipeline. Their intermediate timing differs while their defined architectural results agree. [R15, R35]

This package supplies an original teaching machine, G16 v1, to keep the first implementation bounded. It is not RISC-V and is not binary-compatible with any commercial ISA. The assigned RISC-V reading is used to compare a real published ISA's precision with the teaching contract, not to imply G16 compliance. Learners choosing an actual RISC-V subset must document the exact supported instructions, encodings and omissions independently.

G16 v1: a complete small contract

Instructions and register values are 16 bits. There are eight registers r0 through r7; r0 always reads zero and writes to it are discarded. The program counter is a 16-bit word address. Instruction storage and data RAM are separate 256-word spaces in the supplied execution environment. Each memory word is 16 bits. Arithmetic wraps modulo 65536. Instruction addresses advance by one word, not two bytes.

Supported instructions are NOP, LDI, ADD, SUB, LD, ST, BZ, JMP and HALT. LDI sign-extends a nine-bit immediate. ADD and SUB use three registers. LD and ST add a signed six-bit offset to a base register, wrap that calculation to 16 bits, then require the result to be in data addresses 0 through 255. BZ branches relative to the following instruction when the selected register is zero. JMP uses an eight-bit absolute word address. Reserved bits and unsupported encodings fault.

An illegal instruction, invalid fetch address or invalid data address stops execution without retiring the faulting instruction or changing registers/data memory. HALT retires once, advances PC by one word and stops. There are no privilege levels, interrupts, caches, byte loads, multiplication or standard compiler ABI in this baseline. An instruction-level model step is not a physical clock-frequency claim.

Construct the datapath

A simple single-cycle conceptual datapath fetches the instruction, decodes fields, reads registers, computes an arithmetic result or address, accesses idealized data memory if needed, selects writeback and computes the next PC. The longest active path limits a real single-cycle implementation's clock period. A load can include instruction fetch, decode, register read, address addition, memory read and writeback before the capture edge.

The supplied RTL baseline exposes ideal combinational instruction/data inputs. That is a teaching memory assumption, not a promise about FPGA block RAM. The memory system and testbench provide data according to that assumption. Lesson 34.3 replaces the instantaneous picture with a latency-aware protocol; it is required work, not an optional explanation.

Worked program and encoding

The program below forms 5+7, stores 12 at data word 3, loads it into r4 and halts:

```
LDI r1, 5       # 0x1205
LDI r2, 7       # 0x1407
ADD r3,r1,r2    # 0x2650
ST  r3,r0,3     # 0x5603
LD  r4,r0,3     # 0x4803
HALT           # 0xF000
```

For ADD, bits 15:12 contain opcode 2, bits 11:9 destination 3, bits 8:6 source 1, bits 5:3 source 2 and bits 2:0 zero. Thus 0x2000+0x0600+0x0040+0x0010=0x2650. The store uses the register field as its data source rather than a destination. The exact field interpretation belongs in the ISA table.

At retirement of the ADD, the trace records old PC=2, instruction=0x2650, register-write enabled, destination=3, value=12 and next PC=3. At retirement of the store, it records a memory write of 12 to word 3 and no register write. This trace is more diagnostic than a final “program passed” message.

Independent evidence and unsupported behavior

Write the reference interpreter from the semantic table and the RTL from the datapath plan. Do not share the decoder implementation between them. An assembler is another potential source of correlated bugs; include hand-encoded constants such as those above to check it independently.

Test writes to r0, negative immediates, signed branch displacement, arithmetic wrap, data-address limits, reserved bits and illegal opcodes. A machine that runs one friendly program is not yet a correct implementation of even this small contract. Keep the scope modest and the claims exact.

Practical checkpoint

Lab 34.1: G16 architectural model and datapath. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R35: selected base integer instructions; R15: processor design; R31: computer architecture projects.

Core quiz - original program questions / 6 points

34.1-C1. How does an ISA differ from a microarchitecture?

34.1-C2. What limits a simple single-cycle clock period?

34.1-C3. Why avoid claiming full ISA compliance for a small subset?

Transfer check - new application questions / 4 points

34.1-T1. Why does G16 ADD r3,r1,r2 encode as 0x2650?

34.1-T2. What must happen when LD r0 uses an out-of-range address in the G16 contract?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 34.2 - Pipelining, forwarding, stalls and branch recovery

Learning outcome: Preserve architectural behavior while overlapping instructions.

Game-engineering connection: Pipelining overlaps work from different instructions. Correct overlap requires ownership, valid bits and dependency rules, not merely extra registers between blocks.

Throughput is not the same as latency

Suppose a nonpipelined operation takes T time and a k-stage pipeline divides its logic into stage delays. The pipeline period is limited by the slowest stage plus register and timing overhead, not by T/k automatically. It may complete one instruction per cycle after filling under ideal conditions, while one instruction still takes multiple cycles to retire. Hazards and variable-latency memory reduce the ideal throughput. [R15]

For a conventional illustrative five-stage arrangement, use fetch, decode/register-read, execute, memory and writeback. The G16 baseline is not already this pipeline. The learner may instead build a smaller explicit-valid pipeline model, provided it preserves architectural results and meets the required dependency, load-use, branch and memory tests.

Valid bits separate work from bubbles

Every pipeline register carrying an instruction needs a valid indicator or equivalent ownership state. A bubble means no instruction is present; it is not simply a convenient bit pattern that happens to decode as NOP. On stall, preserve the stage data and validity that still own work. On flush, clear validity for instructions that must not execute, and suppress all their architectural side effects.

Data and control must travel together: destination register, operation kind, address, store data, exception state and program counter need the same timing interpretation as the associated result. Adding a data register without delaying its write-enable can pair one instruction's result with another instruction's destination.

Data, structural and control hazards

A read-after-write hazard occurs when a consumer needs a producer's result before ordinary register-file writeback makes it visible. Forwarding selects a newer in-flight result when it is available at the required point. Compare destination and source registers only for valid, writing instructions, and do not create dependencies on discarded writes to r0.

A load-use hazard occurs when a consumer follows a load whose data is not yet available for the consumer's execute stage. Under a common five-stage timing model with load data available after memory and forwarding into execute, one bubble is needed for an immediately dependent consumer. That count is a property of the chosen stage timing and memory latency, not a universal law.

Structural hazards occur when instructions compete for a resource, such as a single memory port. Control hazards arise when the next PC depends on a branch or jump not yet resolved. A simple correct design may stall fetch until branch resolution. A more aggressive design can fetch speculatively and flush wrong-path instructions. Either must prevent wrong-path stores and register writes.

Worked dependency and branch cases

Consider LDI r1,5 followed immediately by ADD r2,r1,r1. A pipeline reading the old register file without forwarding or a stall might compute zero. A forwarding selection should prefer the youngest matching producer that precedes the consumer, not an older value farther down the pipeline.

Now LD r1,r0,0 followed by ADD r2,r1,r1. If the load result is only available after the memory stage, executing the add too soon is wrong even though the address calculation was available earlier. Forwarding the load's address as though it were its value is a common diagnostic failure.

For control, use BZ r0,+1 followed by ST r1,r0,4 and then HALT. The branch is always taken because r0 is zero, so the store at the next sequential address must not modify memory word 4. A final-register-only comparison can miss that wrong-path memory side effect; retirement and memory-event checks must include it.

Define global priority before coding

Reset, exception/fault, branch recovery and stalls can arrive together. Choose a priority that preserves precise architectural behavior. A branch flush may need to override a hold on younger stages, while an older unresolved memory operation may prevent retirement. For the initial in-order design, stop and reason about the oldest instruction rather than trying to optimize every overlap immediately.

The reference interpreter has no pipeline bubbles. Compare at architectural retirement, not at equal cycle numbers. Record both cycle and retirement index so a mismatch can be localized without demanding identical timing. If the pipeline model cannot yet execute a feature, mark the requirement open rather than filtering that instruction out of the advertised supported subset.

Performance evidence after correctness

Measure completed instructions, cycles, fill/drain cost, stalls by cause and branch recovery cost on identified programs. Calculate CPI as cycles divided by retired instructions under a stated counting window. Report an idealized cycle result as modeled. Without a constrained target timing result, multiplying cycle count by an invented fast clock does not establish a physical speedup.

Practical checkpoint

Lab 34.2: Pipeline-valid and hazard integration. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: pipelined processors; R35: instruction semantics; R32: tracing and debugging.

Core quiz - original program questions / 6 points

34.2-C1. Does pipelining always shorten one instruction's latency?

34.2-C2. Why must flushed instructions suppress writes?

34.2-C3. What is a load-use hazard?

Transfer check - new application questions / 4 points

34.2-T1. Why can forwarding the ALU result of a load fail to solve its immediate load-use dependency?

34.2-T2. A taken branch skips a store, but the final registers match the reference. Is that enough to validate branch recovery?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 34.3 - Memory interfaces, caches and protection boundaries

Learning outcome: Design memory behavior as a protocol rather than an instantaneous array.

Game-engineering connection: Real memory is a participant in a protocol, not a magical array. Requests may wait, responses may arrive later, and device writes can have side effects that must occur exactly once.

Separate request acceptance from completion

A request channel can use req_valid and req_ready. Acceptance occurs at an edge where both are true. The sender must keep the request payload stable while valid is asserted and ready is low. A separate response channel reports completion later. A single-outstanding baseline stores the accepted request identity and does not issue a second one until the first response is consumed.

The CPU must not interpret “still waiting” as permission to repeat a side effect. If a store is accepted once and the response takes five cycles, the memory write still occurs once under the defined model. Holding request-valid high after acceptance can describe another transaction on a later ready edge unless the protocol state prevents it.

Make the memory model explicit

Specify address units, data width, alignment, byte enables if any, allowed outstanding count, read/write ordering, error response and reset/abort behavior. G16 uses 16-bit words and no byte enables in the baseline, but an extension must still define whether an invalid address faults before acceptance or returns an error response afterward.

For a precise in-order machine, a faulting instruction must not commit an architectural register write or store, and younger instructions must not commit ahead of an unresolved older fault. A simulation that returns all loads instantly can hide this issue. The variable-latency lab deliberately forces stalls to expose it.

Cache basics without overclaiming

A cache stores copies of memory blocks. In a direct-mapped cache, address fields select a byte/word within a block, an index and a tag. A valid tag match is a hit. A miss requires a fill or another declared policy. Write-through sends writes onward under its ordering rules; write-back keeps dirty data that must be written to the backing memory when required, including eviction.

For a byte-addressed example with 32-bit addresses, 16-byte blocks and 64 direct-mapped lines, offset width is four bits, index width six and tag width 22. The teaching G16 machine is word-addressed, so this example's bit split must not be copied into G16 without adjusting the address units. Addresses separated by 64*16=1,024 bytes map to the same index and can conflict.

A cache is not an MMU. Address translation and protection are separate mechanisms. A small educational CPU may omit them, but then it does not provide the isolation expected of a protected general-purpose operating system. Likewise, memory-mapped device registers should not be cached as ordinary RAM unless the system contract explicitly supports it. [R15, R35]

Worked exactly-once store

A CPU presents a store to word 3 with value 12. For cycles 0 and 1, req_valid=1 and req_ready=0: no transaction is accepted, and address/data remain stable. At cycle 2, both are 1: one acceptance occurs. The sender moves to WAIT_RESPONSE and removes the request offer for subsequent cycles.

The memory returns a response at cycle 5, which the CPU accepts and uses to retire the store. A write counter attached to the accepted request reports one, not four. If reset occurs after acceptance but before response, the interface contract must say whether the write has already taken effect and how stale responses are rejected. Resetting the scoreboard without considering the memory side effect can conceal a bug.

Latency variation is a verification tool

Randomize readiness and response latency with a reproducible seed, bounded response assumptions and a timeout. A timeout is not proof of deadlock unless the environment was obliged to make progress; it is still valuable diagnostic evidence. Maintain a ledger of issued, accepted, completed and retired transactions, with unique identifiers where needed.

Use adverse cases: repeated reads of one address, writes followed by reads, zero and maximum valid address, invalid addresses, reset while idle and while outstanding, and long response delay. For an optional cache, include two conflicting blocks, dirty eviction and an uncached device access. A cache benchmark that fits forever in one line does not establish correct miss or replacement behavior.

What belongs in this lesson's pass

The latency-aware memory exercise is required. A cache implementation is optional, but any cache that is included must pass conflict and policy tests. Keep the first extension single-outstanding and in order until its handshake is correct. Performance enhancements are valuable only after transaction ownership and architectural effects are reliable.

Practical checkpoint

Lab 34.3: Variable-latency memory and transaction ledger. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: caches, memory and I/O; R35: memory access and selected privilege concepts as an extension.

Core quiz - original program questions / 6 points

34.3-C1. Why is an instantaneous array an incomplete memory model?

34.3-C2. What does a cache tag identify?

34.3-C3. Does a small CPU necessarily need an MMU?

Transfer check - new application questions / 4 points

34.3-T1. A 32-bit byte address uses 16-byte blocks and 64 direct-mapped lines. Give offset, index and tag widths.

34.3-T2. A store request is accepted once, then its response is delayed five cycles. How many times may the memory write occur?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 34.4 - Processor verification, toolchains and architectural evidence

Learning outcome: Demonstrate instruction correctness with reproducible programs and traces.

Game-engineering connection: A processor is verified by observable instruction behavior, including failures and side effects. A blinking display or one successful arithmetic result is not enough.

Compare at the architectural boundary

Retirement is the point at which an instruction's architectural effect becomes committed according to the chosen machine. An instruction-level interpreter can emit a semantic event for each completed instruction. A pipelined or multicycle DUT emits an event only when the corresponding work retires. Compare event sequences, not equal-time register snapshots from unlike microarchitectures. [R15, R32]

A useful trace contains retirement index, cycle, old PC, instruction bits, next PC, register-write enable/destination/value, memory side-effect address/value and fault/halt information. Fields that are not meaningful need an explicit sentinel or enable, not arbitrary stale values that create false mismatches. A faulting instruction in G16 does not retire; record a separate terminal fault event and compare fault state and absence of side effects.

Directed tests define the minimum contract coverage

The original curriculum requires at least twenty directed programs plus a reproducible randomized set. This package includes a directed-case inventory and an instruction-level reference suite to start from. The learner still needs to run the same relevant programs on their implementation and supply their own tests. Executing supplied reference tests alone is not processor implementation evidence.

Cover each instruction, positive and negative immediates, zero-register behavior, dependent arithmetic, wrapping, load/store ordering, address endpoints, branch taken/not taken, negative branch displacement, jump targets, HALT, reserved bits and unsupported opcodes. Include faults that would be hidden by a discarded r0 destination. For a pipeline, add wrong-path writes and dependency chains; for a latency-aware memory system, rerun with stalls.

Random generation must preserve the intended test domain

Generate legal instructions from a specification-aware generator when testing normal execution, and generate deliberately illegal encodings in a separate suite. Keep an instruction budget so a random loop cannot hang the runner forever. A timeout is a distinct result, not a successful HALT. Seed the generator and record initial state, memory and program image.

Avoid a shared bug between generator, assembler and decoder. Some tests should use manually calculated machine words. Differential agreement is strong evidence only when implementations are sufficiently independent and the compared fields actually cover the contract. Two models can agree on an incorrect interpretation copied from the same ambiguous sentence.

Toolchain and binary layout are part of the artifact

The supplied G16 assembler accepts a small documented assembly syntax and writes four-hex-digit words, one instruction word per line, for the simulation image. Labels are resolved to word addresses, and BZ displacement is relative to PC+1. Textual hexadecimal words are not a byte-endianness specification for an external bus. A binary exporter must define byte order separately.

A supported assembled program is enough for the module's compiled-or-assembled requirement. Do not claim that an unmodified C compiler targets G16; it does not have a standard ABI or backend here. A later compiler exercise can target the custom machine under an explicitly defined calling convention. For a real ISA subset, use an appropriately configured toolchain and inspect emitted instructions to ensure it does not introduce unsupported operations. [R35]

Worked first-divergence localization

Suppose the reference retires ADD at PC=2 with r3=12, while the DUT retires r3=5. The preceding LDI traces match. That narrows the search to the ADD's source selection, data hazard or arithmetic path rather than the entire program. Inspect the decoded source indices, pre-execute operands and forwarding selection at that instruction.

If registers match but the memory ledger shows two stores to word 3, the problem may be request acceptance rather than instruction decode. If the first mismatch is next PC on a backward branch, inspect sign extension and the branch base convention. Store the smallest failing program and seed as a permanent regression. Do not merely increase timeouts or remove the failing case.

Evidence quality and independent execution

A reproducible command should assemble/generate images, run simulation, compare traces and return a nonzero status on a mismatch. Record tool versions and parameter choices in machine-readable form. Preserve raw output before summarizing it. A second person should reproduce one normal case, one branch case and one fault case from the README.

The test runner included here distinguishes Python reference execution from external HDL simulation. If Icarus or another simulator is absent, the HDL stage is reported as not run. The package's validation report states what actually executed in the build environment. The learner's final processor review must close their own RTL, pipeline and memory-protocol evidence, not inherit unearned pass marks from the package.

Practical checkpoint

Lab 34.4: Twenty-program processor verification campaign. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R35: exact chosen ISA revision; R32: simulation; R15: software/processor interface.

Core quiz - original program questions / 6 points

34.4-C1. Why compare at retirement rather than arbitrary internal cycles?

34.4-C2. Why inspect compiler-generated instructions?

34.4-C3. What belongs in a reproducible processor failure report?

Transfer check - new application questions / 4 points

34.4-T1. Why compare retirement events rather than register files after the same number of cycles in a reference interpreter and pipeline?

34.4-T2. Does generating random legal programs with the same decoder code used by the DUT provide fully independent verification?

Do not open the key until you have recorded an answer and its reasoning.

Module 35 - FPGA-based game subsystems and integration

Prerequisites: Modules 21, 31-34. Physical deployment is optional; simulated I/O satisfies the core project.

Module project: Gate G: demonstrate a verified RTL game subsystem, such as a sprite compositor, controller interface or bounded vector unit. Include a software reference, interface specification, automated verification and synthesis evidence. A physical board is an extension, not a prerequisite.

35.1 - Memory-mapped I/O, timers, interrupts and DMA. Explain how software controls asynchronous hardware.

35.2 - Framebuffers, scanout and pixel pipelines. Produce timed pixel data with correct storage and scheduling.

35.3 - A small accelerator and its software contract. Measure whether specialized hardware improves a complete operation.

35.4 - Integrated digital-design review and optional board bring-up. Close the loop between specification, verification and implementation.

Lesson 35.1 - Memory-mapped I/O, timers, interrupts and DMA

Learning outcome: Explain how software controls asynchronous hardware.

Game-engineering connection: Game software controls hardware through contracts: registers, accepted requests, completion events and buffer ownership. The interface is part of the design, not an afterthought.

A memory-mapped register is not ordinary RAM

A CPU or host accesses a device register through an address, but reads and writes can have side effects. Reading a queue data register might consume an item; writing a command register might start work; writing a one to a status bit might clear it. Specify access width, alignment, read behavior, write masks, reset value and error behavior for every address. [R15]

The Gate G queue project uses a single-outstanding request/response interface so these effects can be reasoned about clearly. A request is accepted once at req_valid AND req_ready. Its response is then retained until rsp_valid AND rsp_ready. The responder must not repeat the side effect while holding a response. Backpressure on the response channel changes completion time, not the number of enqueues or dequeues.

Timers and interrupt status need priorities

A timer can increment a counter on enabled edges and compare it against a configured threshold. State whether the comparison uses the old or incremented count, whether the timer wraps, how writes interact with ticking, and whether the interrupt is level-sensitive or a pulse. A useful interrupt source latches a pending bit until software acknowledges it.

For a sticky event with write-one-to-clear acknowledgement, a common explicit policy is set wins: pending_next=(pending AND NOT clear) OR event. If clear and a new event occur together, pending remains set. Reversing that priority can lose the new event. A single pending bit records “at least one event,” not an exact event count; multiple arrivals before service can coalesce. Use a counter or queue when every occurrence must be counted.

Interrupt enable usually controls the outgoing interrupt signal, not whether status can latch, but the design must say so. Reading a status register should not clear it unless that destructive-read behavior is intentionally specified. A software read-modify-write on a W1C register can accidentally clear unrelated pending events; the driver should write the intended mask.

DMA is an ownership protocol

Direct memory access lets a device move data without the CPU copying each word in software. It does not remove address validity, lifetime, ordering or coherence requirements. The driver prepares a buffer, establishes the required visibility under the platform's DMA rules, hands ownership to the device, waits for documented completion, then reclaims it. A CPU must not free or modify a device-owned buffer outside the declared sharing contract.

Cache coherence is platform-specific. Volatile access alone is not a universal cache-maintenance or interconnect-ordering mechanism. This volume models ownership and ordering abstractly; it does not provide a production operating-system DMA driver. Physical or OS integration must use the chosen platform's documented APIs and memory model. Keep the scope bounded instead of casting host pointers into imaginary bus addresses.

Worked event race

Assume pending=1 from an earlier timer event. Software writes a clear mask in the same edge that the timer generates a new event. Under set-wins, (1 AND NOT 1) OR 1 = 1. The old event is acknowledged but the new event keeps status asserted. On the next edge, with clear=1 and event=0, pending becomes zero.

For the teaching timer model, define enabled update as next_count=(count+1) modulo 2^W, and event when next_count equals compare. Starting count=0, compare=3 and enable=1, the post-edge counts are 1,2,3 and the event occurs on the third enabled edge. A pre-increment comparison would shift that behavior; both are implementable but only one matches this specification.

Worked queue-register transaction

The capstone DATA register is at byte offset 0x04 and carries a 16-bit queue word in a 32-bit transaction. A write with nonzero upper bits is an error under the supplied project contract. A full-queue write returns an error with no enqueue. An empty-queue read returns an error with no dequeue. This error-return policy differs from a bus that simply stalls until the condition clears; neither behavior should be inferred from the register name.

After a valid enqueue request is accepted, the peripheral produces one response. Holding rsp_ready low for five cycles must not enqueue the word five more times. A FIFO test alone does not establish this adapter property, so the integration scoreboard tracks both bus transactions and queue contents.

Software must test failure paths

Write driver-model tests for invalid offsets, read-only writes, reserved bits, full and empty errors, delayed responses, reset and interrupted work. State which operation owns the data at each step. A register header generated from a map is useful, but the behavioral map and driver tests are what keep software and RTL synchronized.

Practical checkpoint

Lab 35.1: Timer, sticky interrupt and MMIO side effects. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: I/O and interrupts; R35: ordering concepts for the chosen platform; R32: device testbench modeling.

Core quiz - original program questions / 6 points

35.1-C1. Why are device registers not ordinary memory variables?

35.1-C2. When may software reuse a DMA source buffer?

35.1-C3. Does C++ volatile make a shared device driver thread-safe?

Transfer check - new application questions / 4 points

35.1-T1. With pending=1, clear=1 and event=1, what does set-wins W1C logic produce, and why?

35.1-T2. Can holding a device response valid for five cycles justify repeating the original enqueue five times?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 35.2 - Framebuffers, scanout and pixel pipelines

Learning outcome: Produce timed pixel data with correct storage and scheduling.

Game-engineering connection: A framebuffer is storage; scanout is a timed delivery process. A correct image in memory can still produce a broken display when counters, blanking or pipeline alignment are wrong.

Separate storage format from raster timing

A framebuffer's footprint depends on width, height, pixel format, row pitch and buffering count. An unpadded W-by-H image with B bytes per pixel uses W H B bytes. Real systems can pad rows or use tiled/compressed layouts. A double-buffered image uses two separately owned buffers; a front/back pointer swap is not permission to draw into the buffer currently being scanned. [R15]

A simple raster generator counts horizontal pixel periods and vertical lines. Each dimension has an active region plus any declared blanking intervals. If horizontal total is Htotal and vertical total Vtotal, frame frequency is pixel_clock/(Htotal Vtotal). The active pixel count is Hactive Vactive. Blanking still consumes timing periods even when no visible pixel is fetched.

The small timings in the supplied example are educational simulation timings, not a claim of compatibility with an HDMI, DisplayPort or VGA sink. Physical video requires the chosen interface, board circuitry, electrical signaling, clock generation and standard-compliant timing. A color-bar model is not a complete video transmitter.

Define coordinates and boundaries precisely

Let h run from 0 through Htotal-1 and v from 0 through Vtotal-1. Active is h<Hactive AND v<Vactive. At the horizontal terminal count, h wraps to zero and v advances; at the final line, both return to zero. The exact edge that constitutes frame_start must be documented so a buffer swap happens once at the intended boundary.

With registered pixel processing, delay active, sync and coordinates along with the color data. If the color arrives one cycle later but active does not, the last active pixel can appear in blanking and the first can be lost. A full-frame pixel comparison catches this more reliably than inspecting a thumbnail.

Memory demand and starvation

For an unpadded 320-by-240 RGB565 image, storage is 320*240*2=153,600 bytes. Double buffering uses 307,200 bytes. At 60 frames/s, visible pixel reads alone require 9,216,000 bytes/s, or 9.216 decimal MB/s, before bus overhead or competing clients. This average does not prove that a line deadline can be met under burst stalls.

A line buffer can decouple burst memory service from steady pixel delivery, but it adds capacity and scheduling constraints. Define underflow behavior: an error flag, a diagnostic color or another deterministic output, rather than stale unlabelled pixels. A memory interface that averages enough bandwidth can still starve scanout during a long service gap.

Worked educational timing

Choose Hactive=64, Htotal=80, Vactive=48 and Vtotal=60. Each frame has 4,800 pixel periods but only 3,072 active pixels. A modeled pixel clock of 288,000 Hz gives 60 frames/s. That is an arithmetic teaching example, not a physical video mode.

The supplied color-bar reference derives color from active x coordinates and emits black when inactive. A capture routine collects only active pixels in raster order into a PPM image, while the test also checks all 4,800 timing samples. Matching the 3,072 colors alone would not detect an incorrectly long blanking interval.

Now add one registered color stage. At sampled time t, output color corresponds to input coordinates from t-1. The active and boundary signals accompanying that output must be delayed by the same stage or explicitly interpreted as belonging to the earlier coordinate. Write this alignment in a table before inserting the register.

Buffer swapping and sprite composition

For double buffering, accept at most one pending swap request under a declared policy, then change the front buffer at the next defined frame boundary. A request arriving exactly at the boundary needs a specified priority: accepted for that boundary or deferred to the following one. Either can be tested; leaving it unspecified creates nondeterministic expectations between driver and RTL.

A small sprite compositor selects background or sprite pixels using bounded coordinates and transparency rules. Define clipping at negative and out-of-range positions, sprite priority, transparent pixel encoding and arithmetic widths. Unsigned subtraction of a negative coordinate can wrap into a valid-looking large address. Test partially off-screen sprites and every edge, not only a centered sprite.

Verification layers

Check counter periods, active dimensions, sync/blanking conventions, image contents, pipeline alignment, swap timing and underflow behavior separately. Label a generated PPM as a software or simulation capture. It is not a photograph of board operation. This lesson satisfies its core through simulated scanout and captured-image comparison; board deployment remains optional.

Practical checkpoint

Lab 35.2: Raster counters, color bars and boundary-safe swap. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R15: digital-system timing; R32 and R34: timed testbenches; use the selected board vendor's documented video reference only for a physical extension.

Core quiz - original program questions / 6 points

35.2-C1. How many bytes does the example framebuffer require?

35.2-C2. Is active-pixel bandwidth the whole physical interface bandwidth?

35.2-C3. Why synchronize framebuffer swaps?

Transfer check - new application questions / 4 points

35.2-T1. For 64x48 active pixels inside 80x60 total periods at 288 kHz, find frame frequency and active samples per frame.

35.2-T2. Why can delaying RGB by one register while leaving active unchanged corrupt an otherwise correct frame?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 35.3 - A small accelerator and its software contract

Learning outcome: Measure whether specialized hardware improves a complete operation.

Game-engineering connection: Specialized hardware is useful only when its numerical result, data movement and software overhead improve the complete operation that the game actually needs.

Start with a bounded workload and exact semantics

Choose a small operation such as saturating vector addition, fixed-point multiply/accumulate or pixel blending. Define operand width, signedness, rounding, saturation and result order. The reference streaming example performs signed 16-bit saturating addition: inputs are bit patterns representing two's-complement integers, and the result clamps to -32768 through 32767. It is a building block, not a finished accelerator product.

A streaming interface accepts input only when in_valid AND in_ready at a sampling edge. It produces an output transaction when out_valid AND out_ready. If output is blocked, payload and valid remain stable. A one-entry output register may accept a new input when empty or when its existing output is consumed on the same edge. That replacement rule differs from the conservative FIFO's full policy and is explicitly documented.

Pipeline data, validity and numerical policy together

To implement signed saturation, add in a 17-bit signed intermediate. Compare the widened sum against the 16-bit signed endpoints, then clamp or take the low result bits as appropriate. Extending after a 16-bit addition is too late; overflow information has already been lost.

A longer arithmetic pipeline needs enough buffering or backpressure propagation to prevent overwriting an unconsumed result. Simply freezing the last stage while upstream stages continue can drop data. Track transaction identity through the pipeline and compare accepted outputs against accepted inputs, rather than counting raw clock cycles with valid high.

A driver contract includes setup, transfer and completion

An accelerator call might prepare descriptors, validate buffers, transfer operands, start hardware, wait or poll, transfer results and reclaim ownership. The complete time is setup + input movement + device work + output movement + synchronization, with overlap only where demonstrated. A high theoretical arithmetic throughput does not erase these costs.

Compare against a competent software implementation with the same numerical contract and result quality. Small batches may favor software because fixed overhead dominates. Large batches may become limited by memory bandwidth rather than arithmetic. CPU cache state, DMA coherence, batching and contention can change the result; report the controlled conditions and which values are modeled rather than measured. [R15, R33]

Worked break-even analysis

Use a deliberately illustrative model: a software path costs 2 microseconds plus 8 ns per element; a device path costs 30 microseconds plus 3 ns per element including the modeled transfer cost. Hardware is faster when 30 us + 3N ns < 2 us + 8N ns. Rearranging gives 28,000 ns < 5N ns, so N>5,600. At N=5,600 they tie; the smallest integer winning batch is 5,601 under this model.

At N=1,000, software costs 10 us and device 33 us: the accelerator loses. At N=100,000, software costs 802 us and device 330 us: the modeled operation speedup is about 2.43. These are made-up planning coefficients, not benchmark measurements or claims about a specific GPU/FPGA.

If the operation accounts for 20% of end-to-end application time and this portion speeds up fourfold, Amdahl's-law ideal total speedup is 1/(0.8 + 0.2/4) = 1.17647 before additional overhead. A fast kernel may have modest game-level benefit. Use measured workload fractions in a real project rather than assuming the hotspot occupies the whole frame.

Verify order under backpressure

Feed boundary operands such as 32767+1, -32768-1, opposite-sign cancellation and zero. Randomize input gaps and output readiness while retaining pending input transactions. Maintain a sequence of expected saturating results at accepted input edges, then consume it only at accepted output edges. Test reset while an output is blocked; under the supplied flush policy, pending work is discarded and must not reappear after reset.

A numerical mismatch, duplicate output, missing output and unstable stalled output are different failure classes. Count all of them. Add a timeout only with a declared progress condition, such as output-ready eventually becoming true. Throughput measurements should state the accepted-input count, accepted-output count, cycle window and fill/drain treatment.

The capstone need not be the fastest design

A well-verified queue with a software interface or a modest arithmetic block is a valid Gate G project. The point is to close the specification-to-evidence loop. More ambitious hardware/software optimization belongs in Volume 8; do not replace this volume's verification work with speculative speedup claims.

Practical checkpoint

Lab 35.3: Streaming arithmetic and end-to-end cost. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R33: synthesis and arithmetic mapping; R34: scoreboard verification; R15: parallelism and I/O.

Core quiz - original program questions / 6 points

35.3-C1. What total time follows from halving the example 20 microsecond compute phase?

35.3-C2. What does backpressure accomplish?

35.3-C3. Why report latency and throughput separately?

Transfer check - new application questions / 4 points

35.3-T1. Software costs 2 us+8N ns and hardware 30 us+3N ns. What is the smallest integer N for a strict modeled hardware win?

35.3-T2. If 20% of application time speeds up 4x, what is the ideal total speedup before new overhead?

Do not open the key until you have recorded an answer and its reasoning.

Lesson 35.4 - Integrated digital-design review and optional board bring-up

Learning outcome: Close the loop between specification, verification and implementation.

Game-engineering connection: The final review asks whether another engineer can understand, reproduce and challenge the design. A small subsystem with complete evidence is stronger than an impressive demo with hidden assumptions.

Assemble a claim-evidence chain

Gate G is the undergraduate digital-design capstone. Implement one bounded game-related RTL subsystem: a command queue with processor-style interface, a sprite compositor, a fixed-point vector block or an equivalently scoped alternative. The separate module-level processor exercise remains required even when the capstone centers on another block. Reuse module work; the gate is not an extra unrelated project added to the workload.

For every major claim, name the supporting artifact. “No reordering” should point to a sequence property and tests. “One side effect per request” should point to an accepted-transaction monitor and response-stall tests. “Synthesizable” should point to an actual synthesis report. “Meets this board's timing” requires target-specific implementation timing evidence, not a generic synthesis script.

Required layers of evidence

Begin with the interface and numerical specification, including clocks, resets, widths, address units, legal operations, error behavior and ownership. Keep a software reference independent from the implementation. Include directed and randomized automated tests, assertions, meaningful cover goals or functional coverage, a reset/CDC review and synthesis reports.

Test boundaries that apply to the chosen block: empty/full, simultaneous operations, backpressure, pointer wrap, overflow/rounding, invalid accesses and reset while work is pending. Document inapplicable items with a reason rather than silently leaving cells blank. For example, a pure single-clock integer queue has no fixed-point rounding requirement, but still needs data-width and overflow-of-occupancy reasoning.

Reproduce from a clean directory

A reproducible handoff names the command sequence, tools, versions, top modules, parameters, seeds and expected output files. Include raw logs and machine-readable results when practical. Do not make another person reverse-engineer a path from your shell history. Keep generated reports separate from hand-authored interpretation, and avoid checking in unlabelled stale outputs as though they belong to the final source revision.

An archive checksum helps establish which files were reviewed. It does not prove correctness. An automated suite that runs only the reference model also does not establish RTL correctness. The package's supplied code and tests are learning aids, not an already passed learner capstone.

Worked review: queue plus MMIO

The recommended project combines the parameterized FIFO with a single-outstanding register adapter, status, flush control and a sticky event interrupt. Software can enqueue or dequeue one 16-bit command, read occupancy, flush queued work and acknowledge an empty-to-nonempty event. The Gate G handbook provides a complete register contract and milestone plan.

A critical integration test fills the queue, attempts one extra enqueue and confirms an error response with unchanged contents. It then stalls response-ready after a valid dequeue and confirms only one queue item was removed. Finally it resets while a response is pending and checks the documented cancellation rule. A FIFO-only test would not cover those adapter-level behaviors.

For synthesis, compare at least two supported depths under the same generic flow or a justified target flow. Interpret changes in inferred storage and control logic. No FPGA board is required to complete the core gate, but an actual synthesis run is required. Mark physical timing, power and board operation as not measured when that is the truth.

Optional board bring-up is a separate controlled extension

Before any physical work, obtain the exact board revision and schematic, verify power and I/O voltage compatibility, review pin assignments and reset/clock sources, and use a mentor-approved low-voltage procedure. Do not copy a pin file from a similar-looking board or connect unknown external voltages. No mains equipment disassembly or battery-pack work belongs in this course.

Bring up one observable function at a time: clock/reset sanity, a slow status indication, interface transaction, then the subsystem. Keep a rollback/recovery path for configuration mistakes and stop when electrical assumptions are uncertain. A working FPGA demonstration does not establish ASIC readiness; fabrication, test, physical signoff and reliability introduce additional requirements.

Oral defense and live change

Explain a waveform with pre-edge inputs and post-edge state. Trace a nonblocking assignment example without running it. Derive setup and hold as different inequalities. Explain one failed test and why its repair addresses the root cause. Then make a bounded change, such as depth three to five or a new status bit, predict affected tests and rerun them.

The reviewer should be able to distinguish your reasoning from supplied examples and AI-generated assistance. Record assistance and source provenance, but assess your ability to explain and modify the system independently. Passing means the evidence supports a bounded engineering claim, not that every possible hardware design topic has been mastered.

Practical checkpoint

Lab 35.4: Gate G integration review and reproducible handoff. Complete the independent prediction, procedure, acceptance checks and deliberate-failure exercise in the workbook. Keep raw evidence and distinguish the model or tool layer actually run.

Assigned study

R32-R34: repeatable simulation and synthesis; R15: system integration; handbook safety review for physical extensions.

Core quiz - original program questions / 6 points

35.4-C1. Why are unit tests insufficient for integration?

35.4-C2. What must be checked before wiring a peripheral?

35.4-C3. Does a successful FPGA demonstration prove an ASIC is ready to fabricate?

Transfer check - new application questions / 4 points

35.4-T1. Can a sprite-compositor capstone replace the required Module 34 processor exercise?

35.4-T2. A generic synthesis run succeeds and a board later displays an image. What stronger claims still need separate evidence?

Do not open the key until you have recorded an answer and its reasoning.

Reference directory

Primary-source readings support the technical claims and tool workflows. Original program IDs are preserved. Added references use the V7- prefix. Public documentation was checked for this edition on September 28, 2026; installed tool versions must be recorded separately. No external textbooks, vendor manuals or paid course materials are bundled.

R10 - 6.003 Signals and Systems

MIT OpenCourseWare. Sampling, convolution, transforms and systems.

https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

R15 - 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R30 - 6.002 Circuits and Electronics

MIT OpenCourseWare. Circuits, transistors and laboratory reasoning.

https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R31 - Nand to Tetris

Noam Nisan and Shimon Schocken. Computer construction projects; review the authors' license terms.

https://www.nand2tetris.org/

R32 - Verilator user guide

Verilator project. RTL simulation and linting; match supported SystemVerilog subset.

https://verilator.org/guide/latest/

R33 - Yosys documentation

YosysHQ. Synthesis and technology mapping; stable installed release preferred over a development snapshot.

https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R34 - cocotb documentation

cocotb project. Python-based HDL verification; simulator and version compatibility must be checked.

https://docs.cocotb.org/en/stable/

R35 - RISC-V ISA specifications

RISC-V International. Instruction-set reference; record the exact ISA version and implemented extensions.

https://docs.riscv.org/reference/isa/

R40 - Precalculus, second edition

OpenStax / Rice University. Algebra, functions and trigonometry bridge before calculus.

https://openstax.org/details/books/precalculus-2e

R50 - SymbiYosys documentation

YosysHQ. Formal hardware verification, safety, liveness and cover properties.

https://yosyshq.readthedocs.io/projects/sby/en/latest/

V7-R01 - UG949: Clock Domain Crossing

AMD. Domain crossing and implementation review; record the selected vendor-tool revision.

https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Clock-Domain-Crossing

V7-R02 - Command-line flags

Icarus Verilog. SystemVerilog language selection and command-line usage; package runner selects -g2012.

https://steveicarus.github.io/iverilog/usage/command_line_flags.html

V7-R03 - Documentation directory

ngspice project. Use the manual matching the locally installed version; toy models are not device characterization.

https://ngspice.sourceforge.io/docs.html

V7-R04 - Timing model

cocotb project. Scheduling phases and stable sampling in Python testbenches.

https://docs.cocotb.org/en/stable/timing_model.html

Reading boundary

G16 v1 and GQ-1 are original teaching contracts in this package, not standards defined by the references. Their exact behaviors are established by the authored specifications. Numerical examples are worked from their stated assumptions, not measured device results. Public RISC-V documents are comparative reading; no RISC-V compliance is claimed.